

category: dotnet-general

tags: [wsl, git-bash, msys, dpkg-deb, dotnet-sdk, process-invocation, linux-deploy]

severity: high

created: 2026-07-25

updated: 2026-07-25

project-origin: 7.LinuxDeployTool

Git Bash for Windows KHÔNG phải Linux — tool cần lệnh Linux thật (dpkg-deb, apt, ...) PHẢI chạy qua WSL

Tình huống gặp phải

Tool `Kztek.LinuxDeployTool` (WinForms) gọi `bash scripts/linux-deb/build-deb.sh` để đóng gói `.deb` cho các solution .NET/Avalonia. `BashRunner.cs` detect và chạy qua Git Bash (MSYS).

Triệu chứng / Lỗi

Lỗi — xem log
[stderr] `scripts/linux-deb/build-deb.sh: line 128: dpkg-deb: command not found`

Nguyên nhân gốc rễ (Root Cause)

`dpkg-deb` là công cụ đóng gói Debian — chỉ tồn tại trong **Linux/WSL thật**. Git Bash for Windows (MSYS/Git for Windows) chỉ là môi trường POSIX giả lập chạy TRÊN Windows, không phải một distro Linux — nó không bao giờ có `dpkg-deb`, `apt`, hay bất kỳ tool Debian-specific nào, bất kể detect đúng `bash.exe` (Git Bash thật, không nhầm WSL stub) bao nhiêu đi nữa.

Lịch sử nhầm lẫn để tái diễn: một lỗi TRƯỚC ĐÓ

khiến `%LOCALAPPDATA%\Microsoft\WindowsApps\bash.exe`

(chỉ là symlink stub trỏ `wsl.exe`) bị detect nhầm là Git Bash, gây lỗi path `/e/...` (format MSYS)

không tồn tại trong WSL (WSL cần `/mnt/e/...`). Fix cho lỗi path đó để khiến người sửa kết luận sai "phải tránh WSL hoàn toàn, chỉ dùng Git Bash" — nhưng đó là kết luận SAI khi script cần tool Linux thật. Root cause thực sự của lần đó là **path format sai + tool không phải WSL thật (là stub)**, không phải "không nên dùng WSL".

Giải pháp

1. Xác định: script/lệnh có gọi tool Linux-only nào không (`dpkg-deb`, `apt`, `systemctl`, `chroot`, container tools, ...)? Nếu có → **BẮT BUỘC** chạy qua WSL thật, Git Bash không đủ.

2. Dùng `wsl.exe -e bash -c "cd '<path>' && <cmd>"` (KHÔNG dùng Git Bash `bash.exe` cho các lệnh này).
3. Convert path Windows → format WSL: `C:\foo\bar` → `/mnt/c/foo/bar` (không phải `/c/foo/bar` kiểu MSYS).
4. Nếu lệnh cũng cần `dotnet` (hoặc tool .NET khác): distro WSL phải có `dotnet` SDK nằm trong PATH của **non-interactive shell** — tức symlink vào thư mục hệ thống như `/usr/local/bin/dotnet`, KHÔNG chỉ export trong `~/.bashrc`/`~/.profile`, vì `wsl.exe -e bash -c "..."` không load các file đó (không phải interactive/login shell).

```
```bash
```

```
sudo ln -sf $HOME/.dotnet/dotnet /usr/local/bin/dotnet
```

5. Verify bằng đúng cách gọi sẽ dùng trong code (không verify bằng shell tương tác của user, vì PATH có thể khác):

```
```bash
```

```
wsl.exe -e bash -c "command -v dotnet; command -v dpkg-deb"
```

Áp dụng lại (How to reuse)

- Khi thấy `<tool>: command not found` từ 1 process được Windows app spawn qua "bash" → luôn hỏi:
"tool này có phải Linux-native không?" (`dpkg-deb`, `apt`, `systemd`, `mount`, `chroot`...) → nếu có, đừng cố tìm cách chạy qua Git Bash, chuyển thẳng sang `wsl.exe`.
- Trước khi "fix" một detection bug (nhằm WSL stub là Git Bash) bằng cách LOẠI TRỪ HÃN WSL — dừng lại hỏi: "lệnh phía sau có cần tool chỉ có trong Linux không?". Nếu có, exclude sai đối tượng — cái cần sửa là path-format handling, không phải né tránh WSL.
- Khi 1 process gọi qua `wsl.exe -e bash -c "..."` báo `command not found` dù user xác nhận "tool đã cài rồi" (chạy `command -v` trong terminal tương tác của họ ra kết quả) → nghi ngay PATH khác nhau giữa interactive shell (có `.bashrc`) và non-interactive `-c` invocation — yêu cầu họ symlink vào `/usr/local/bin` thay vì chỉ dựa vào PATH export trong rc file.

Chú ý / Cạm bẫy (Gotchas)

- ⚠ Build cross-project reference (ProjectReference) qua DrvFs (`/mnt/<ổ Windows>/...`) đôi khi lỗi tạm thời `CSC : error CS0006: Metadata file '.../ref/Xxx.dll' could not be found` ở lần build đầu
— nghi race condition/caching khi nhiều project build song song ghi file qua 9p/DrvFs. Retry lần 2 thường qua vì file đã tồn tại. Nếu lặp lại nhiều → cân nhắc `-maxcpucount:1` cho bước publish.
- ⚠ `wsl --install` / cài package cần `sudo` — sudo yêu cầu password tương tác, KHÔNG thể chạy non-interactive qua `sudo -n`. Agent không tự nhập password được — phải đưa lệnh cho user tự chạy

trong terminal WSL thật.

- ⚠ Không assume tất cả máy dev đều có sẵn đúng SDK version cần thiết trong WSL — kiểm tra `dotnet --list-sdks` trước, đừng ép cài đúng bản trùng khớp target framework nếu không cần thiết (SDK mới hơn build được project target cũ hơn nhờ backward-compatible + NuGet runtime packs).

Tham chiếu

- Project liên quan: `7.LinuxDeployTool`
(`Kztek.LinuxDeployTool/Services/BashRunner.cs`)
- Xem thêm: `.claude/GOTCHAS.md` G001 trong project `7.LinuxDeployTool`